

Kev and CLM

Difference	Kev (small models)	CLM
Inputs	State, question and options together	State and actions encoded separately
Trainable	LoRA + pointer head	Projection heads only
Features	Encoder behavior changes	LLM features stay fixed

Our lab follows Kev's LoRA + pointer-head recipe.

Key and CLM: pros and cons

Criterion	Key	CLM
Strength	Adapts encoder features Small released models	Reuses action embeddings Cheap repeated head training
Tradeoff	Backpropagates through the encoder for LoRA	Depends on fixed features No joint token interaction
Lab fit	Main lab recipe Full base stage is public	Useful comparison Fixed-feature head training

Accuracy depends on the backbone, data and task.

Training cost

Work	Key LoRA	CLM heads
Encoder	Forward + backward during each training run	Extract fixed features once for each text
Repeat runs	Repeat encoder compute for every experiment	Reuse feature cache Train heads only
Budget	Encoder activations plus adapter optimizer	Feature extraction cost plus head optimizer

Compare total cost: feature extraction + all training runs.

Serving cost and model size

Factor	Kev	CLM
Each new state	Encode state and options Score pointer outputs	Encode state + question Score cached actions
Useful cache	Shared state prefix across questions	Independent action vectors across changing states
BF16 weights*	0.8B: ~1.6 GB 4B: ~8 GB	8B reference: ~16 GB Two paths can share weights

*Parameter count $\times$ 2 bytes; excludes heads, cache and runtime.

Performance evidence

Published result	Setup	What it supports
CLM: 36-44 ms	RTX 4090; cached actions 1-1,024 candidates	Low added cost as the action set grows
Kev-4B: 12 ms	RTX PRO 6000 One question; median	Fast serving in that llama.cpp setup
Kev vs CLM	No matched Pac-Man comparison established	Measure skill, latency and memory together

These reports do not establish a Kev-versus-CLM speed ranking.

Models supported by llama.cpp

Model	Readout	Useful strength	Main tradeoff
Julia-1 144M	Option-marker head	Small; multilingual 50+ languages	Limited knowledge/reasoning Private training pipeline
Laya 421M	Option-marker head	Small; designed for specialization	Fully fine-tuned encoder English root: 512 tokens
Kev-4B	Pointer head	Adaptable features Open training code	LoRA changes features Options encoded each step
lev 4B	Vocabulary label logits	Existing LM readout Many label codes	LoRA; two choice orders Extra head not converted
OpenJev 27B	Vocabulary label logits	Text and images Agent decisions	Larger model Non-commercial weights

How llama.cpp scores decisions

POST /v1/systemone: Choice, Score and Noul

GGUF metadata selects the model's readout

Kev's LoRA is merged before quantization

No generated answer tokens to parse

CLM needs its own serving implementation

How SGLang scores decisions

/v1/decisions and /v1/systemone are merged

Chat-model label logits; no generated tokens

One prompt per question; normal prefix caching

Checked: Qwen3.8-27B and Qwen3.5-35B-A3B

Native Intern-Decision support is an open PR

A fair Kev-versus-CLM comparison

Use the same boards, labels and legal actions

Time cold and warm caches on one device

Measure accuracy, captures, dots and latency

Record training time and peak memory

Match backbones to isolate the architecture

Our Kev-4B Pac-Man lab

Train Pac-Man with fixed-code ghosts

Start from Qwen3.5-4B-Base

Train fresh LoRA and a pointer head

Complete four general stages before class

Fine-tune your own checkpoint for moves

Our 90-minute route

- 0-10 Compare architectures and play
- 10-20 Inspect the four general training stages
- 20-30 Review Pac-Man player labels
- 30-60 Fine-tune LoRA and the pointer head
- 60-80 Compare cost and player decisions
- 80-90 Explain and submit

Colab runtime for this lab

Target: RTX PRO 6000 Blackwell, 96 GB

PyTorch 2.8 / CUDA 12.8 with BF16

Required: FLA and CUDA causal convolution

Verify Kev loss and backward before training

Use checkpointing and fused AdamW

CP0: what Pac-Man can observe

Maze walls and remaining dots

Player position and heading

Ghost positions and headings

Turn number and ghost movement rule

CP1: score a legal player move

Pack state, question and legal options

Read decision-token and option-token features

Score each option with the pointer head

Read probabilities by direction name

Apply the move through the game code

What a probability tells us

Select the highest-scoring legal action

Inspect competing directions

Compare calibration on labelled examples

Confidence is not maze success

A legal move can still be strategically poor

CP2: prepare player labels

64 train, 16 development, 16 evaluation

Inspect three boards and annotate legal moves

Starter labels follow a stated heuristic

Keep all edits in the training partition

Keep evaluation closed until the model is fixed

Kev-4B training stages

- 1 Decisions: 12,576 records $\times$ 2 epochs
- 2 Dates/evidence: 1,425 + 2,000 replay
- 3 Documents: 5,219 + 2,000 replay
- 4 Skills/devtools: 11,320 + 4,000 replay
- 5 Pac-Man: 64 labelled boards $\times$ 2 epochs

CP3: 30 minutes of fine-tuning

5 min Inspect labelled moves and the loss

20 min Fine-tune your skills checkpoint

5 min Inspect and save the adapter/head

Update LoRA and the pointer head together

Keep original base matrices unchanged

Check what training changed

Trainable: LoRA and pointer-head parameters

Original base matrices remain fixed

Encoder features change through LoRA

Track loss and live memory in TensorBoard

Save recovery weights during every stage

CP4: compare the same boards

Fix skills and Pac-Man checkpoints

Score the same 16 evaluation boards

Compare teacher accuracy and captures

Save paired predictions by board ID

Include a deterministic rules baseline

CP4: compare cost and gameplay

Use identical starts and fixed-code ghosts

Compare dots collected and game turns

Measure end-to-end decision latency

Record actual training time and peak memory

Record Colab compute units consumed

CP5: explain and submit

Explain one changed move and one mistake

Show which parameters received gradients

Save all five adapters and pointer heads

Submit paired predictions and training curves

Record runtime and actual compute-unit use

The community starting point

Source: [codaaiteam/jev-pacman](#), MIT

Original demo: human player, Jev ghosts

Our design: learned player, fixed-code ghosts

Use the same maze and rules for every model

No ROM setup for this browser clone